

**ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ**



**BÁO CÁO PROJECT CUỐI HỌC PHẦN
THIẾT KẾ SỐ VÀ VI XỬ LÝ - 2526I_ELT3290_68
CHỦ ĐỀ: VENDING MACHINE
(MÁY BÁN HÀNG TỰ ĐỘNG)**

**Giảng viên hướng dẫn: TS.Vũ Quốc Tuấn
Sinh viên thực hiện: Vũ Hải Phong - 23020856
Lê Nguyễn - 23020852**

HÀ NỘI – 2026

Mục Lục

I) GIỚI THIỆU CHUNG	3
1.1. Vai trò của bộ xử lý trong hệ thống số.....	3
1.2. Ý nghĩa của mô hình	3
1.3. Mục tiêu của project.....	4
II) CƠ SỞ LÝ THUYẾT	4
2.1. Tổng quan về kiến trúc, Ưu điểm – Nhược điểm.....	4
2.2. Datapath của bộ xử lý.....	5
2.3. Nguyên lý điều khiển	5
III) THIẾT KẾ CÁC MODULE CHỨC NĂNG.....	7
3.1. Module Điều khiển (VM_controller).....	7
3.2. Module Đơn vị Số học (alu8).....	12
3.3. Module Tra cứu giá (price_lookup).....	14
3.4. Module Chuyển đổi BCD (bin8_to_bcd).....	16
3.5. Module Giải mã LED 7 đoạn (seven_seg_decoder).....	18
4.6. Module Khôi dữ liệu (VM_datapath)	20
IV) THIẾT KẾ KIẾN TRÚC TỔNG THỂ.....	25
4.1. Sơ đồ khối hệ thống của project.....	25
4.2. Phân tích luồng thực thi lệnh.....	27
4.3 Top Module vending_machine	29

I) GIỚI THIỆU CHUNG

Dự án tập trung vào việc thiết kế và mô phỏng một hệ thống máy bán hàng tự động (Vending Machine) sử dụng ngôn ngữ mô tả phần cứng SystemVerilog. Hệ thống được thiết kế theo cấu trúc phân cấp, tách biệt rõ ràng giữa khối điều khiển và khối tính toán dữ liệu.

1.1. Vai trò của bộ xử lý trong hệ thống số

Trong hệ thống số này, "bộ xử lý" được triển khai dưới dạng kiến trúc **FSMD (Finite State Machine with Datapath)**. Vai trò của nó được chia thành hai phần cốt lõi:

- **Bộ điều khiển (Controller - VM_controller):** Đóng vai trò là "não bộ". Nó chịu trách nhiệm quản lý trạng thái của hệ thống (như chờ máy, nạp tiền, chọn hàng, kiểm tra số dư). Nó nhận tín hiệu từ người dùng (nút bấm, cảm biến xu) và các phản hồi từ khối dữ liệu để đưa ra các quyết định logic, điều phối các thành phần khác hoạt động đúng trình tự.
- **Khối dữ liệu (Datapath - VM_datapath):** Đóng vai trò là "cơ bắp" và "máy tính". Khối này thực hiện các phép toán số học (cộng tiền nạp, trừ tiền hàng thông qua ALU), lưu trữ giá trị (số dư, mã sản phẩm), đếm thời gian (Timer) và thực hiện thuật toán chia tiền thừa (Greedy Algorithm).

1.2. Ý nghĩa của mô hình

Mô hình máy bán hàng tự động này mang lại nhiều ý nghĩa thực tiễn và học thuật:

- **Mô phỏng thực tế:** Hệ thống tái hiện quy trình giao dịch tự động phổ biến, bao gồm xử lý mệnh giá tiền (5k, 10k, 20k, 50k), tra cứu giá sản phẩm dựa trên mã số, và quản lý thời gian chờ (timeout) để đảm bảo an toàn giao dịch.
- **Ứng dụng thuật toán tối ưu:** Việc tích hợp **Thuật toán Tham lam (Greedy Algorithm)** để tính toán số lượng tờ tiền trả lại tối ưu nhất cho khách hàng minh chứng cho khả năng kết hợp giữa toán học và phần cứng.
- **Giao diện trực quan:** Mô hình tích hợp bộ giải mã LED 7 thanh và tín hiệu LED trạng thái, giúp người dùng dễ dàng theo dõi số dư, mã hàng hoặc số tiền còn thiếu.

1.3. Mục tiêu của project

- Dự án được thực hiện nhằm đạt được các mục tiêu sau:

+ **Quản lý giao dịch:** Theo dõi số dư tiền nạp và trừ tiền dựa trên giá của sản phẩm được chọn.

+ **Kiểm soát trạng thái:** Điều hướng người dùng qua các giai đoạn: Nghỉ (IDLE) -> Nạp tiền (CHARGE) -> Chọn hàng (SELECT) -> Trả hàng (DISPENSE) -> Trả tiền thừa (RETURN).

+ **Giao tiếp người dùng:** Hiển thị thông tin trực quan lên các LED 7 đoạn (số dư, mã hàng, số tiền còn thiếu)

II) CƠ SỞ LÝ THUYẾT

2.1. Tổng quan về kiến trúc, Ưu điểm – Nhược điểm

- Hệ thống máy bán hàng được thiết kế theo kiến trúc **FSMD (Finite State Machine with Datapath)**. Đây là kiến trúc chuẩn trong thiết kế hệ thống số phức tạp, nơi logic điều khiển và các phép toán số học được tách biệt.

- **Ưu điểm:**

+ **Tính module hóa cao:** Dễ dàng bảo trì hoặc nâng cấp riêng biệt khối điều khiển (FSM) hoặc khối tính toán (Datapath).

+ **Khả năng mở rộng:** Có thể dễ dàng thêm các loại mệnh giá tiền mới hoặc danh mục sản phẩm mà không cần thay đổi toàn bộ cấu trúc máy trạng thái.

+ **Độ tin cậy:** Việc tách biệt giúp việc mô phỏng và kiểm lỗi (debug) trở nên rõ ràng, đặc biệt là khi xử lý các tín hiệu thời gian thực như nút bấm và cảm biến.

- **Nhược điểm:**

+ **Độ trễ:** Việc trao đổi tín hiệu giữa Controller và Datapath có thể mất một vài chu kỳ clock (latency).

+ **Tốn tài nguyên:** So với các máy trạng thái đơn giản (Ad-hoc logic), kiến trúc FSMD sử dụng nhiều register và đường dây nối (interconnect) hơn.

2.2. Datapath của bộ xử lý

Khối Datapath (VM_datapath) đóng vai trò lưu trữ và xử lý các dữ liệu định lượng. Các thành phần chính bao gồm:

1. Hệ thống Register (Thanh ghi):

- current_balance (8-bit): Lưu trữ số dư hiện tại của khách hàng.
- current_product_code (5-bit): Lưu trữ mã sản phẩm do người dùng nhập từ keypad.

2. Khối ALU (Arithmetic Logic Unit):

- Thực hiện phép cộng khi nạp tiền: $\text{Balance} = \text{Balance} + \text{Coin_in}$.
- Thực hiện phép trừ khi mua hàng: $\text{Balance} = \text{Balance} - \text{Price}$.
- Xuất Các cờ trạng thái is_neg và is_zero được trả về để Controller quyết định bước tiếp theo

3. Khối Price Lookup (Bảng tra cứu giá):

- Đóng vai trò như một bộ nhớ ROM nhỏ, chuyển đổi từ mã sản phẩm (5 bit) sang đơn giá (8 bit) tương ứng.

4. Khối Timer (Bộ định thời):

- Sử dụng bộ đếm integer timer_count để đếm xung clock. Khi đạt ngưỡng $\text{TIMEOUT_VAL} = 15$, tín hiệu tmr_out sẽ kích hoạt trạng thái trả lại tiền tự động.

5. Thuật toán thối tiền (Greedy Algorithm):

- Sử dụng các phép chia (/) và chia lấy dư (%) để tính toán số lượng tờ tiền các mệnh giá 50k, 20k, 10k, 5k sao cho tổng số tờ tiền là ít nhất.

2.3. Nguyên lý điều khiển

- Bộ điều khiển (VM_controller) hoạt động dựa trên mô hình Máy trạng thái hữu hạn (FSM). Nguyên lý cốt lõi là dựa vào các đầu vào từ người dùng và các cờ trạng thái từ Datapath để chuyển đổi giữa các trạng thái logic:

+ **S_IDLE (Trạng thái nghỉ):** Chờ đợi tín hiệu coin_valid. Khi có tiền nạp vào, máy sẽ xóa các dữ liệu cũ (clr_code = 1) và bắt đầu giao dịch.

+ **S_CHARGE (Nạp tiền):** Cho phép người dùng nạp nhiều lần. Đồng thời kích hoạt Timer (tmr_en = 1). Nếu quá thời gian quy định mà không có thao tác, máy chuyển sang S_RETURN.

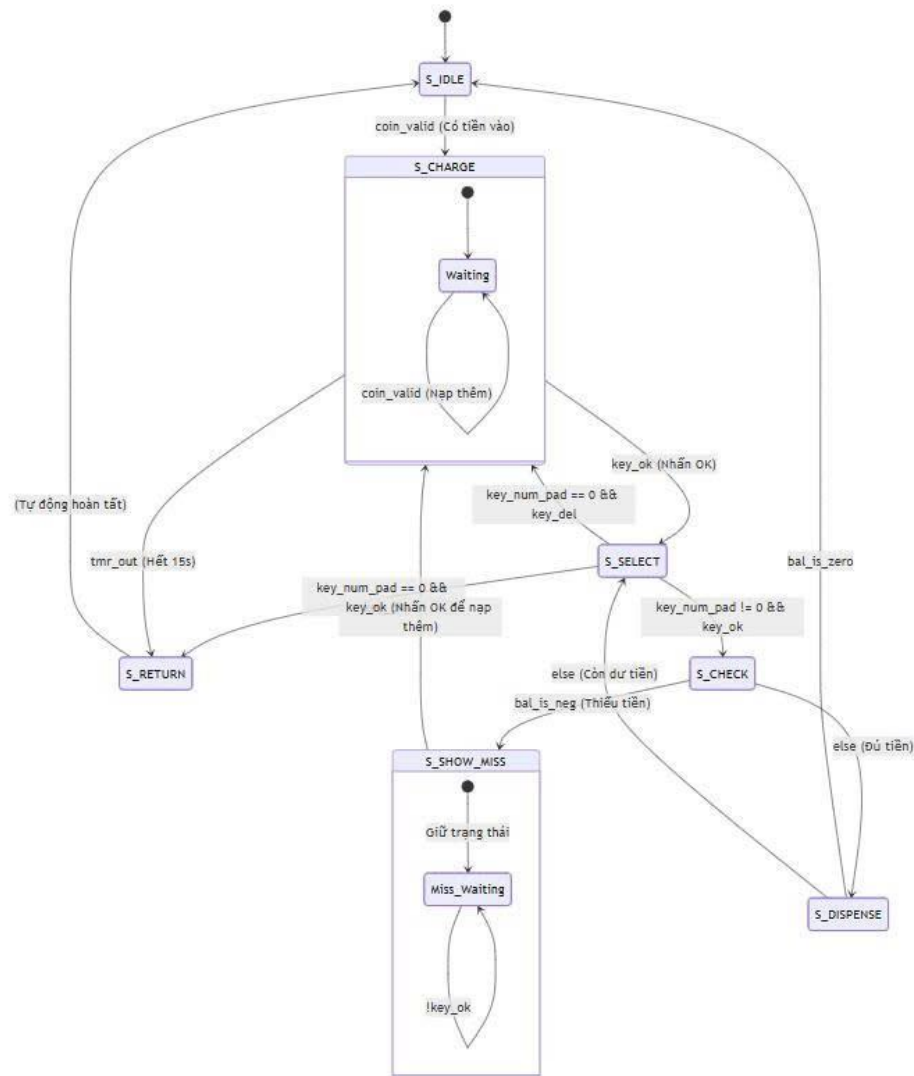
+ **S_SELECT (Chọn hàng):** Ghi nhận mã sản phẩm từ bàn phím khi người dùng thông qua (ld_code = 1)

+ **S_CHECK (Kiểm tra):** Thực hiện phép tính Balance - Price thông qua ALU
(alu_sub_mode = 1, src_sel = 2'b01)

+ **S_SHOW_MISS (Báo thiếu):** Nếu kết quả phép trừ là âm, máy hiển thị số tiền còn thiếu trên LED 7 thanh và yêu cầu nạp thêm.

+ **S_DISPENSE (Nhả hàng):** Nếu đủ tiền, máy kích hoạt tín hiệu dispense (dispense = 1) , trừ tiền thật vào số dư và quay lại trạng thái nạp tiền (nếu còn dư) hoặc kết thúc.

+ **S_RETURN (Trả tiền):** Trạng thái cuối cùng, kích hoạt khối tính toán tiền thừa (calc_change = 1) và xóa sạch các thanh ghi để chuẩn bị cho lượt giao dịch mới.



III) THIẾT KẾ CÁC MODULE CHỨC NĂNG

3.1. Module Điều khiển (VM_controller)

- Module này được thiết kế theo mô hình **FSM Moore** kết hợp một vài đặc tính của **Mealy** (để phản ứng nhanh với tín hiệu coin_valid).

- **Logic Trạng thái:** Sử dụng kiểu dữ liệu enum để định nghĩa 7 trạng thái : S_IDLE, S_CHARGE, S_SELECT, S_CHECK, S_SHOW_MISS, S_DISPENSE, S_RETURN\$

- **Cơ chế chuyển đổi:**

+ Xử lý đồng bộ: Trạng thái chỉ chuyển tại cạnh dương của xung clk.

+ Xử lý tổ hợp: Tính toán trạng thái tiếp theo (next_state) dựa trên đầu vào.

- **Các tín hiệu điều khiển then chốt:**

+ ld_balance: Cho phép nạp giá trị mới vào thanh ghi tiền.

+ src_sel: Chọn nguồn dữ liệu cho ALU (Chọn từ coin_in khi nạp tiền, chọn từ price_wire khi thanh toán).

+ alu_sub_mode: Xác định phép tính là Cộng hay Trừ.

- **Nguyên lý:** Sử dụng mô hình Máy trạng thái hữu hạn (**FSM**). Nó hoạt động dựa trên sự thay đổi của các tín hiệu đầu vào để quyết định hệ thống đang ở bước nào trong quy trình bán hàng.

- **Cấu trúc:** Sử dụng một khối always @(posedge clk) để chuyển trạng thái và một khối always @(*) để quyết định các tín hiệu điều khiển.

+ *Ví dụ:* Khi ở trạng thái S_CHECK, nếu tín hiệu bal_is_neg từ Datapath báo về là 1, nó sẽ lập tức chuyển trạng thái sang S_SHOW_MISS.

File VM_controller

```
1 module VM_controller (
2     input logic clk,
3     input logic rst_n,
4
5     // Inputs
6     input logic coin_valid,
7     input logic key_ok,
8     input logic key_del,
9
10    // từ Datapath
11    input logic bal_is_neg,
12    input logic bal_is_zero,
13    input logic code_is_zero,
14    input logic tmr_out,
15
16    // Control Outputs
17    output logic ld_balance,
18    output logic clr_balance,
19    output logic ld_code,
20    output logic clr_code,
21    output logic alu_sub_mode,
22    output logic [1:0] src_sel,
23
24    output logic tmr_en,
25    output logic tmr_rst,
26    output logic calc_change,
27
28    // Outputs trạng thái
29    output logic dispense,
30    output logic is_show_miss_state,
31    output logic is_select_state,
32    output logic is_charge_state,
33    output logic is_return_state
34 );
35
36 typedef enum logic [2:0] {
37     S_IDLE,
38     S_CHARGE,
39     S_SELECT,
40     S_CHECK,
41     S_SHOW_MISS,
42     S_DISPENSE,
43     S_RETURN
44 } state_t;
45
46 state_t current_state, next_state;
47
48 // Edge Detection Logic
49 // Tạo xung key_ok_pulse và key_del_pulse chỉ bằng 1 trong 1 chu kỳ clock
50 logic key_ok_prev, key_del_prev;
51 logic key_ok_pulse, key_del_pulse;
52
53 always_ff @(posedge clk or negedge rst_n) begin
54     if (!rst_n) begin
55         key_ok_prev <= 1'b0;
56         key_del_prev <= 1'b0;
57     end else begin
58         key_ok_prev <= key_ok;
59         key_del_prev <= key_del;
60     end
61 end
62
63 assign key_ok_pulse = key_ok && !key_ok_prev;
64 assign key_del_pulse = key_del && !key_del_prev;
65
66 //State Memory
67 always_ff @(posedge clk or negedge rst_n) begin
68     if (!rst_n) current_state <= S_IDLE;
69     else current_state <= next_state;
70 end
```

```

72 //Next State Logic
73 always_comb begin
74     next_state = current_state;
75
76     case (current_state)
77     S_IDLE: begin
78         if (coin_valid) next_state = S_CHARGE;
79     end
80
81     S_CHARGE: begin
82         if (key_ok_pulse) next_state = S_SELECT;
83         else if (tmr_out) next_state = S_RETURN;
84     end
85
86     S_SELECT: begin
87         // Dừng _pulse cho các điều kiện chuyển trạng thái
88         if (code_is_zero && key_del_pulse) begin
89             next_state = S_CHARGE; // Quay lại nạp tiền
90         end
91         else if (code_is_zero && key_ok_pulse) begin
92             next_state = S_RETURN; // Hủy mua -> Trả tiền
93         end
94         else if (!code_is_zero && key_ok_pulse) begin
95             next_state = S_CHECK; // Đã chọn mã -> Kiểm tra tiền
96         end
97         if (tmr_out) next_state = S_RETURN;
98     end
99
100     S_CHECK: begin
101         // State này tự động chuyển, không cần nút bấm
102         if (bal_is_neg) next_state = S_SHOW_MISS;
103         else next_state = S_DISPENSE;
104     end
105
106     S_SHOW_MISS: begin
107         // Sửa: Dừng key_ok_pulse để quay lại nạp thêm
108         if (key_ok_pulse) next_state = S_CHARGE;
109         else if (tmr_out) next_state = S_RETURN;
110     end
111
112     S_DISPENSE: begin
113         next_state = S_RETURN;
114     end
115
116     S_RETURN: begin
117         next_state = S_IDLE;
118     end
119
120     default: next_state = S_IDLE;
121 endcase
122 end
123
124 // Output Logic
125 always_comb begin
126     ld_balance = 0; clr_balance = 0;
127     ld_code = 0; clr_code = 0;
128     alu_sub_mode = 0; src_sel = 2'b00;
129     tmr_en = 0; tmr_rst = 0;
130     calc_change = 0; dispense = 0;
131
132     is_show_miss_state = 0;
133     is_select_state = 0;
134     is_charge_state = 0;
135     is_return_state = 0;
136
137     case (current_state)
138     S_IDLE: begin
139         clr_code = 1;
140         if (coin_valid) begin
141             ld_balance = 1;
142             src_sel = 2'b00;
143             clr_balance = 0;
144             tmr_rst = 1;
145         end else begin
146             clr_balance = 1;
147         end
148     end
149 end

```

```

151     S_CHARGE: begin
152         is_charge_state = 1;
153         tmr_en = 1;
154         if (coin_valid) begin
155             ld_balance = 1;
156             src_sel = 2'b00;
157             alu_sub_mode = 0;
158             tmr_rst = 1;
159         end
160     end
161
162     S_SELECT: begin
163         is_select_state = 1;
164         ld_code = 1; // Cho phép cập nhật mã hàng từ bàn phím
165         tmr_en = 1;
166     end
167
168     S_CHECK: begin
169         src_sel = 2'b01;
170         alu_sub_mode = 1; // Phép trừ kiểm tra (Balance - Price)
171     end
172
173     S_SHOW_MISS: begin
174         is_show_miss_state = 1;
175         tmr_en = 1; // Vẫn đếm timer khi hiển thị thiếu tiền
176     end
177
178     S_DISPENSE: begin
179         dispense = 1;
180         ld_balance = 1; // Cập nhật lại balance (trừ tiền)
181         src_sel = 2'b01;
182         alu_sub_mode = 1;
183         tmr_rst = 1; // Reset timer cho trạng thái sau
184     end
185
186     S_RETURN: begin
187         is_return_state = 1;
188         calc_change = 1; // Kích hoạt tính tiền thừa
189     end
190 endcase
191 end
192
193 endmodule

```

File VM_controller_tb.sv

```

1  `timescale 1ns/1ps
2
3  module VM_controller_tb();
4      logic clk, rst_n;
5      logic coin_valid, key_ok, key_del;
6      logic bal_is_neg, bal_is_zero, code_is_zero, tmr_out;
7
8      // Outputs
9      logic ld_balance, clr_balance, ld_code, clr_code, alu_sub_mode, tmr_en, tmr_rst, calc_change;
10     logic [1:0] src_sel;
11     logic dispense, is_show_miss_state, is_select_state, is_charge_state, is_return_state;
12
13     VM_controller dut (.*);
14     always #5 clk = ~clk;
15
16     initial begin
17         // Khởi tạo
18         clk = 0; rst_n = 0;
19         coin_valid = 0; key_ok = 0; key_del = 0;
20         bal_is_neg = 0; bal_is_zero = 1; code_is_zero = 1; tmr_out = 0;
21
22         #20 rst_n = 1;
23         $display("[0ns] System Reset");
24
25         // Kích bản 1: Nạp tiền (IDLE -> CHARGE)
26         #20 coin_valid = 1;
27         #10 coin_valid = 0;
28         $display("[Case 1] Coin Inserted: State should be S_CHARGE");
29

```

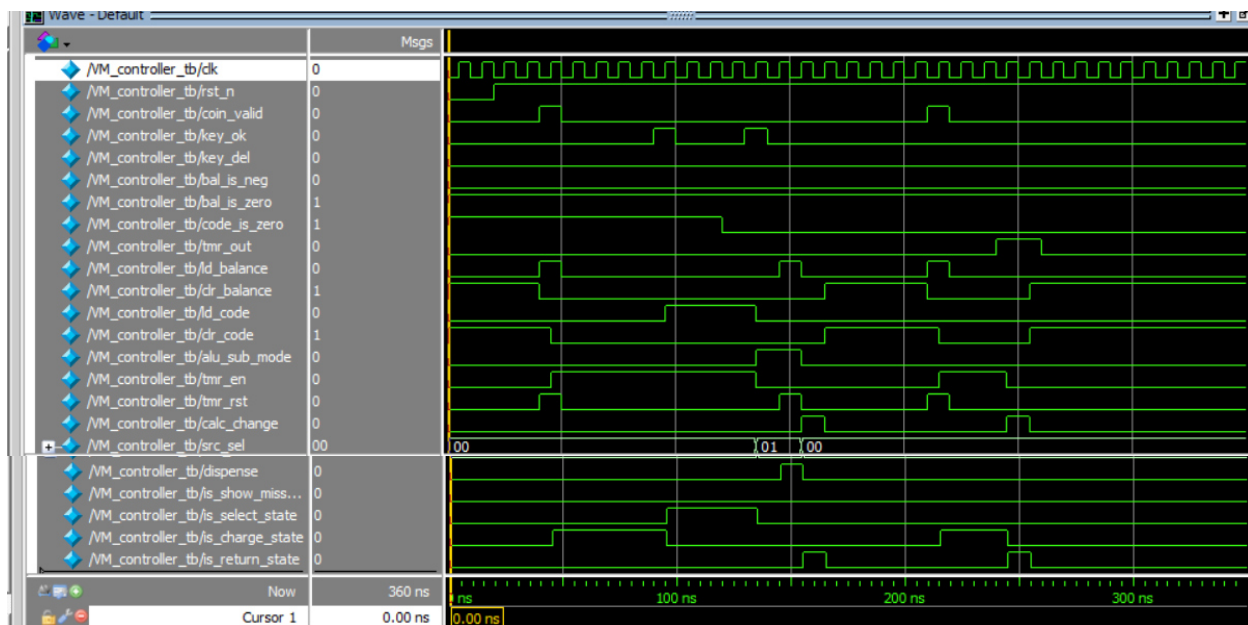


```

30      // Kịch bản 2: Chọn hàng (CHARGE -> SELECT)
31      #40 key_ok = 1; #10 key_ok = 0;
32      $display("[Case 2] OK Pressed: State should be S_SELECT");
33
34      // Kịch bản 3: Kiểm tra tiền đủ (SELECT -> CHECK -> DISPENSE)
35      #20 code_is_zero = 0; // Đã nhập mã
36      #10 key_ok = 1; #10 key_ok = 0; // Nhấn OK để mua
37      #10 bal_is_neg = 0; // Giả lập Datapath báo đủ tiền
38      $display("[Case 3] Purchase: State should go S_CHECK -> S_DISPENSE");
39
40      // Kịch bản 4: Trả lại IDLE
41      #40;
42      $display("[Case 4] End Transaction: State should be S_IDLE");
43
44      // Kịch bản 5: Timeout (CHARGE -> RETURN)
45      #20 coin_valid = 1; #10 coin_valid = 0;
46      #20 tmr_out = 1; // Giả lập quá thời gian
47      #20 tmr_out = 0;
48      $display("[Case 5] Timeout Occurred: State should go to S_RETURN");
49
50      #100 $finish;
51  end
52  endmodule

```

Mô phỏng ModelSim:



[Case 1] Coin Inserted: State should be S_CHARGE

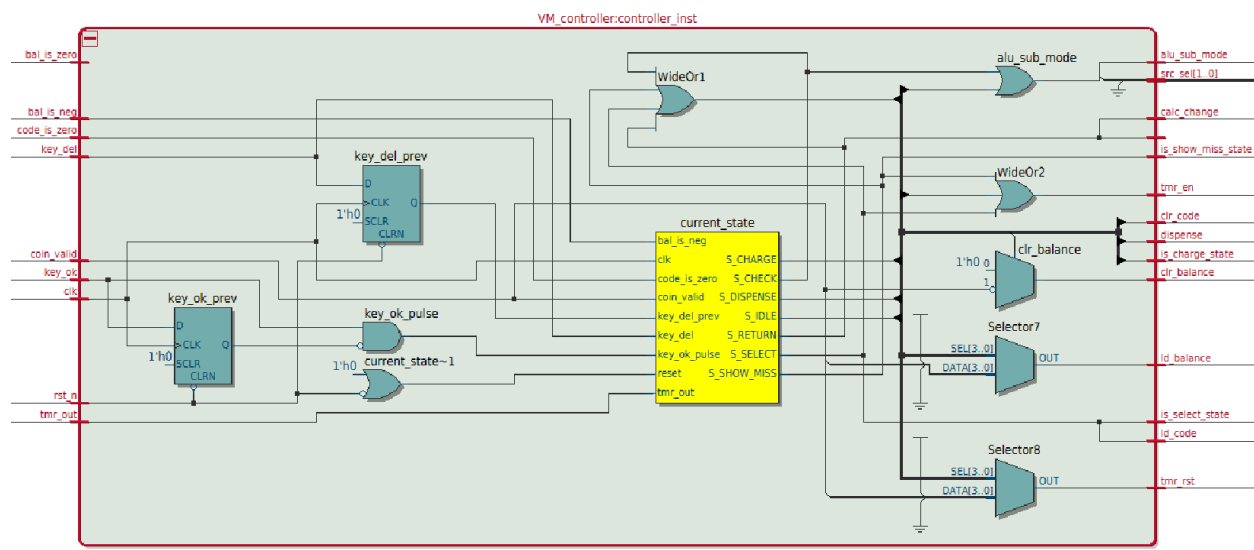
[Case 2] OK Pressed: State should be S_SELECT

[Case 3] Purchase: State should go S_CHECK -> S_DISPENSE

[Case 4] End Transaction: State should be S_IDLE

[Case 5] Timeout Occurred: State should go to S_RETURN

Cấu trúc RTL Viewer thu được trên Quartus:



3.2. Module Đơn vị Số học (alu8)

- Module này xử lý các phép tính 8-bit với độ chính xác cao nhờ kỹ thuật mở rộng bit.

+ Kỹ thuật 9-bit: Để bắt được lỗi thiếu tiền (Underflow), ALU sử dụng một biến tạm 9-bit: $\text{temp_result} = \{1'b0, a\} - \{1'b0, b\}$

+ Cờ trạng thái:

- is_neg: Nếu bit thứ 9 ($\text{temp_result}[8]$) bằng 1, nghĩa là kết quả phép trừ bị âm -> báo về Controller để chuyển sang trạng thái S_SHOW_MISS.
- is_zero: Kiểm tra nếu kết quả bằng 0 để tắt các trạng thái nạp tiền.

- **Nguyên lý:** Là một bộ tính toán tổ hợp. Nó không có bộ nhớ, chỉ nhận đầu vào và cho ra kết quả ngay lập tức.

- **Xử lý bit dấu:** Để biết tiền thiếu hay đủ, ALU thực hiện phép trừ Balance - Price. Nó sử dụng một biến tạm 9-bit. Nếu bit cao nhất (bit thứ 9) là 1, điều đó có nghĩa là phép trừ bị âm (thiếu tiền), từ đó bật cờ is_neg.

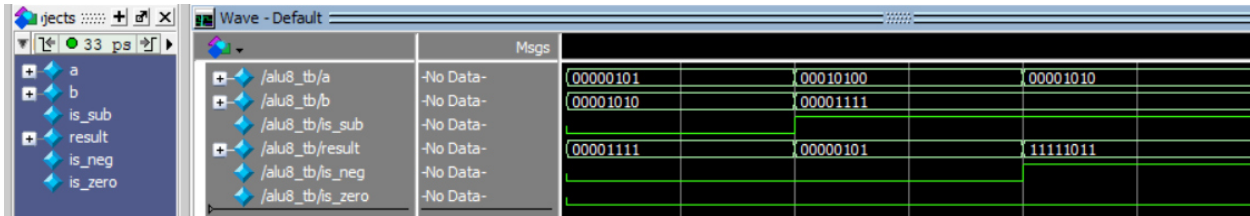
File alu8.sv

```
1 module alu8 (  
2     input logic [7:0] a,          // Balance hiện tại  
3     input logic [7:0] b,          // Coin in hoặc Price  
4     input logic is_sub, // 0: Add, 1: Sub  
5     output logic [7:0] result,  
6     output logic is_neg, // Cờ báo âm (cho trường hợp thiếu tiền)  
7     output logic is_zero // Cờ báo bằng 0  
8 );  
9     logic [8:0] temp_result; // 9 bit để bắt dấu/tròn  
10  
11 always_comb begin  
12     if (is_sub)  
13         temp_result = {1'b0, a} - {1'b0, b};  
14     else  
15         temp_result = {1'b0, a} + {1'b0, b};  
16     end  
17  
18     assign result = temp_result[7:0];  
19     assign is_neg = temp_result[8];  
20     assign is_zero = (result == 8'd0);  
21  
22 endmodule
```

File alu8_tb.sv

```
1 module alu8_tb();  
2     logic [7:0] a, b;  
3     logic is_sub;  
4     logic [7:0] result;  
5     logic is_neg, is_zero;  
6  
7     alu8 dut (.*);  
8  
9     initial begin  
10         $display("Testing ALU8...");  
11         // Case 1: 5 + 10 = 15  
12         is_sub = 0; a = 8'd5; b = 8'd10;  
13         #10; $display("Add: 5 + 10 = %d (Neg: %b, Zero: %b)", result, is_neg, is_zero);  
14  
15         // Case 2: 20 - 15 = 5  
16         is_sub = 1; a = 8'd20; b = 8'd15;  
17         #10; $display("Sub: 20 - 15 = %d (Neg: %b)", result, is_neg);  
18  
19         // Case 3: 10 - 15 = -5  
20         is_sub = 1; a = 8'd10; b = 8'd15;  
21         #10; $display("Sub: 10 - 15 = %d (Neg: %b)", result, is_neg);  
22  
23         $finish;  
24     end  
25 endmodule
```

Mô phỏng ModelSim



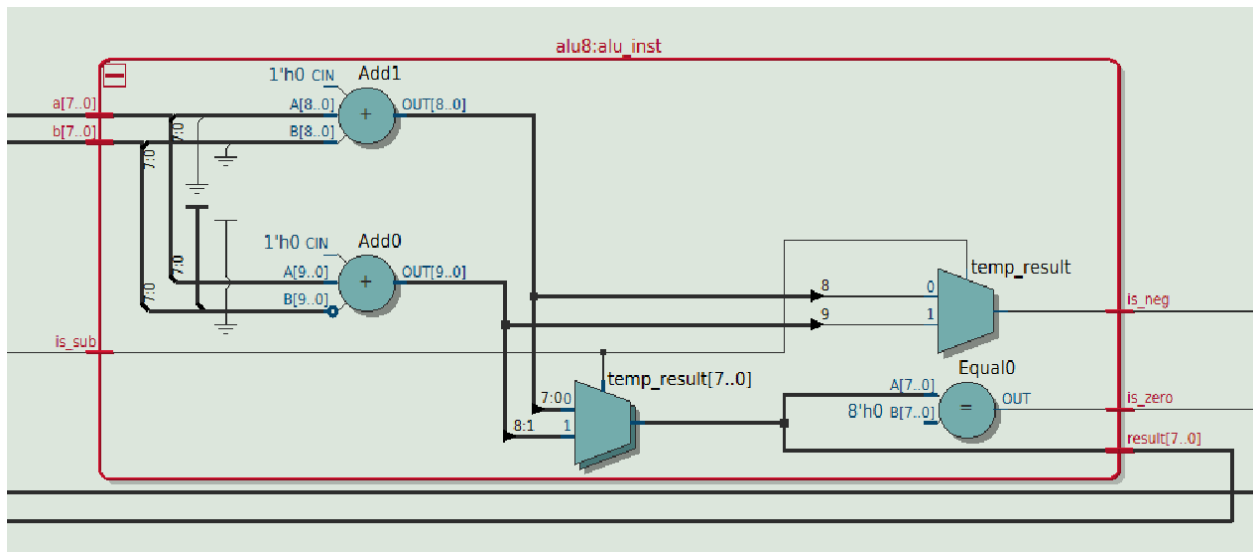
Testing ALU8...

Add: $5 + 10 = 15$ (Neg: 0, Zero: 0)

Sub: $20 - 15 = 5$ (Neg: 0)

Sub: $10 - 15 = 251$ (Neg: 1)

Cấu trúc RTL Viewer thu được trên Quartus:



3.3. Module Tra cứu giá (price_lookup)

- Đóng vai trò như một bộ nhớ ROM nhỏ chứa bảng giá sản phẩm.

+ **Cơ chế:** Nhận mã sản phẩm (5-bit) và xuất ra đơn giá (8-bit).

+ **Logic:** Sử dụng cấu trúc if-else để phân nhóm mã:

- Mã 1 - 5: Giá 5 đơn vị.
- Mã 6 - 10: Giá 10 đơn vị... và tiếp tục cho đến mã 25.

+ **Báo lỗi:** Nếu mã không nằm trong danh sách, tín hiệu is_invalid sẽ được kích hoạt.

+ **Cấu trúc Logic:** Sử dụng lệnh if-else để phân loại mã sản phẩm.

File price_lookup.sv

```
1 module price_lookup (  
2     input logic [4:0] product_code,  
3     output logic [7:0] price,  
4     output logic is_invalid  
5 );  
6     always_comb begin  
7         is_invalid = 1'b0;  
8         price = 8'd0;  
9  
10        if (product_code >= 5'd1 && product_code <= 5'd5) begin  
11            price = 8'd5; // Mã 1-5: Giá 5  
12        end else if (product_code >= 5'd6 && product_code <= 5'd10) begin  
13            price = 8'd10; // Mã 6-10: Giá 10  
14        end else if (product_code >= 5'd11 && product_code <= 5'd15) begin  
15            price = 8'd15; // Mã 11-15: Giá 15  
16        end else if (product_code >= 5'd16 && product_code <= 5'd20) begin  
17            price = 8'd20; // Mã 16-20: Giá 20  
18        end else if (product_code >= 5'd21 && product_code <= 5'd25) begin  
19            price = 8'd25; // Mã 21-25: Giá 25  
20        end else begin  
21            price = 8'd0;  
22            is_invalid = 1'b1;  
23        end  
24    end  
25 endmodule
```

File price_lookup_tb.sv

```
1 module price_lookup_tb();  
2     logic [4:0] product_code;  
3     logic [7:0] price;  
4     logic is_invalid;  
5  
6     price_lookup DUT (.*);  
7  
8     initial begin  
9         product_code = 5'd3; #10; // price = 5  
10        product_code = 5'd22; #10; // price = 25  
11        product_code = 5'd30; #10; // is_invalid = 1  
12        $stop;  
13    end  
14 endmodule
```

Mô phỏng ModelSim

product_code	/price_lookup_tb/product...	No Data-
price	/price_lookup_tb/price	No Data-
is_invalid	/price_lookup_tb/is_invalid	No Data-

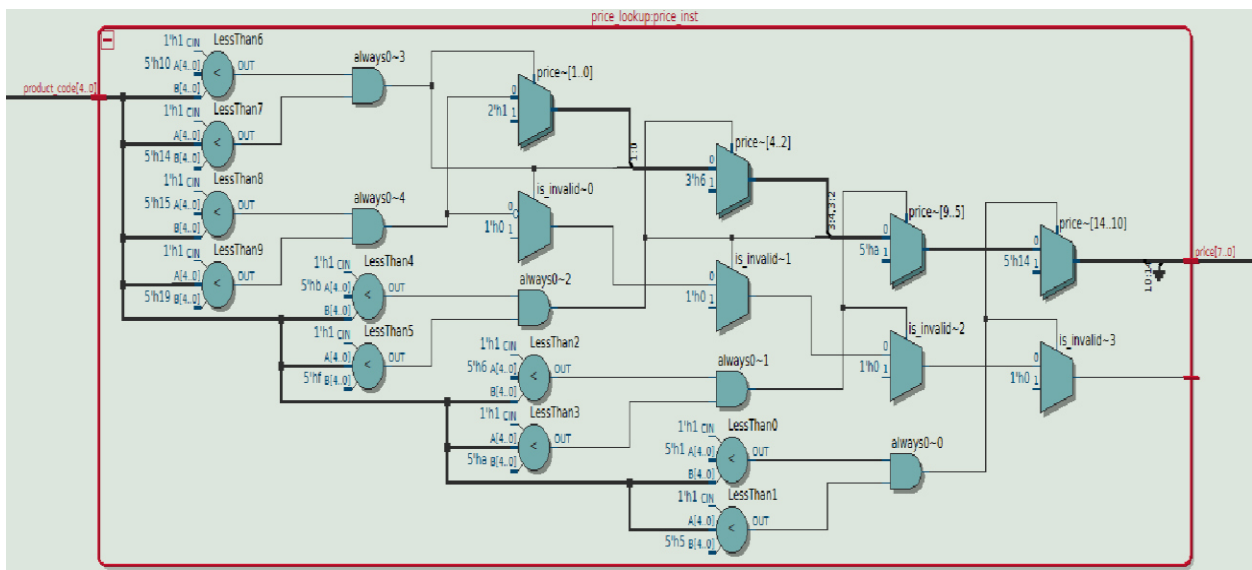
00011	10110	11110
00000101	00011001	00000000

product_code = 00011 (3) → price = 00000101 (giá 5k)

product_code = 10110 (22) → price = 00011001 (giá 20k)

product_code = 11110 (30) → is_invalid = 0 (không có sản phẩm)

Cấu trúc RTL Viewer thu được trên Quartus:



3.4. Module Chuyển đổi BCD (bin8_to_bcd)

- Chuyển đổi dữ liệu từ hệ nhị phân sang mã thập phân để hiển thị lên LED 7 đoạn.

+ **Thuật toán:** Sử dụng phép chia cho 100 và 10 để tách số 8-bit thành 3 chữ số riêng biệt: Hàng trăm (hundreds), Hàng chục (tens), và Hàng đơn vị (ones).

+ **Ý nghĩa:** Giúp người dùng dễ dàng đọc số dư (ví dụ: số nhị phân 01010101 được tách thành số 8 và 5 để hiển thị là 85).

File bin8_to_bcd.sv

```

1  module bin8_to_bcd (
2      input  logic [7:0] bin_in,
3      output logic [3:0] hundreds,
4      output logic [3:0] tens,
5      output logic [3:0] ones
6  );
7
8      always_comb begin
9          hundreds = bin_in / 100;
10         tens      = (bin_in % 100) / 10;
11         ones      = (bin_in % 100) % 10;
12     end
13 endmodule

```

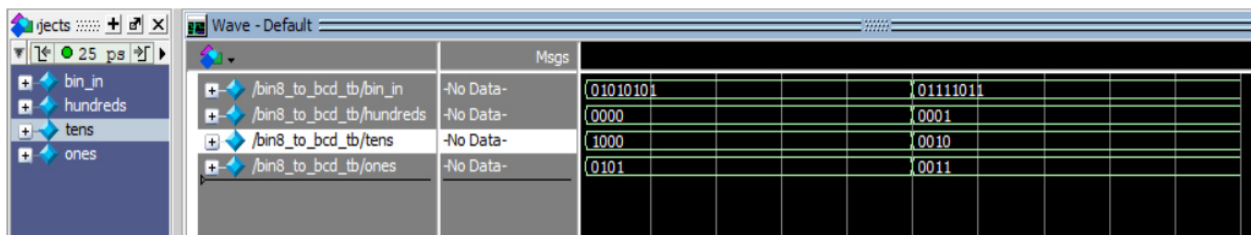
File bin8_to_bcd_tb.sv

```

1  module bin8_to_bcd_tb();
2      logic [7:0] bin_in;
3      logic [3:0] hundreds, tens, ones;
4
5      bin8_to_bcd dut (.*);
6
7      initial begin
8          $display("Testing Binary to BCD...");
9          // Test s 85
10         bin_in = 8'd85; #10;
11         $display("Bin: %d -> Hundreds: %d, Tens: %d, Ones: %d", bin_in, hundreds, tens, ones);
12
13         // Test s 123
14         bin_in = 8'd123; #10;
15         $display("Bin: %d -> Hundreds: %d, Tens: %d, Ones: %d", bin_in, hundreds, tens, ones);
16
17         $finish;
18     end
19 endmodule

```

Mô phỏng ModelSim:



Object	Value	Time
bin_in	85	0 ns
hundreds	0	0 ns
tens	8	0 ns
ones	5	0 ns

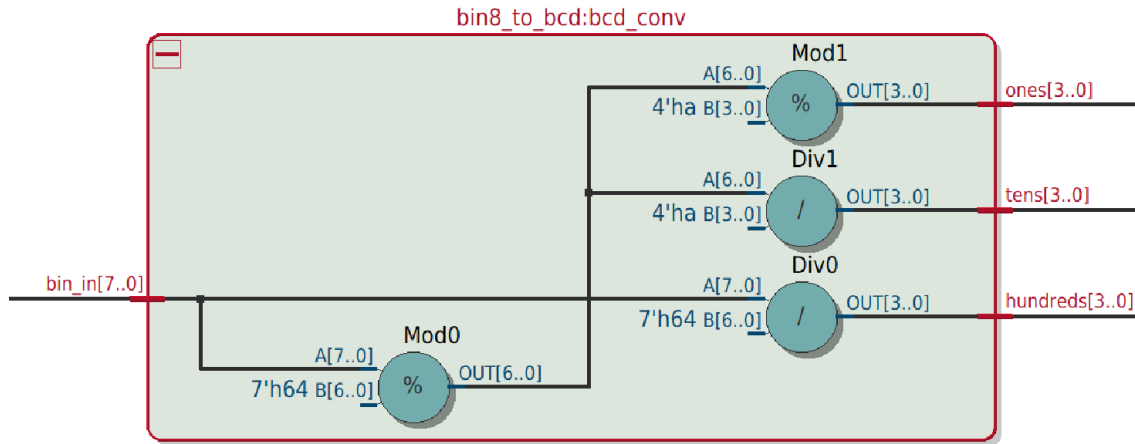
Msgs	Time
/bin8_to_bcd_tb/bin_in	No Data-
/bin8_to_bcd_tb/hundreds	No Data-
/bin8_to_bcd_tb/tens	No Data-
/bin8_to_bcd_tb/ones	No Data-

Testing Binary to BCD...

Bin: 85 -> Hundreds: 0, Tens: 8, Ones: 5

Bin: 123 -> Hundreds: 1, Tens: 2, Ones: 3

Cấu trúc RTL Viewer thu được trên Quartus:



3.5. Module Giải mã LED 7 đoạn (seven_seg_decoder)

- Chuyển đổi mã số 4-bit thành tín hiệu điều khiển các đoạn LED.

+ **Kiến trúc:** Bảng tra cứu (Look-up Table) ánh xạ từ số 0 - 9 sang mã 7-bit đại diện cho các đoạn {a, b, c, d, e, f, g}

+ **Cấu hình:** Thiết kế theo kiểu **Active Low** (mức thấp), nghĩa là bit 0 sẽ làm đoạn LED sáng và bit 1 sẽ làm đoạn LED tắt.

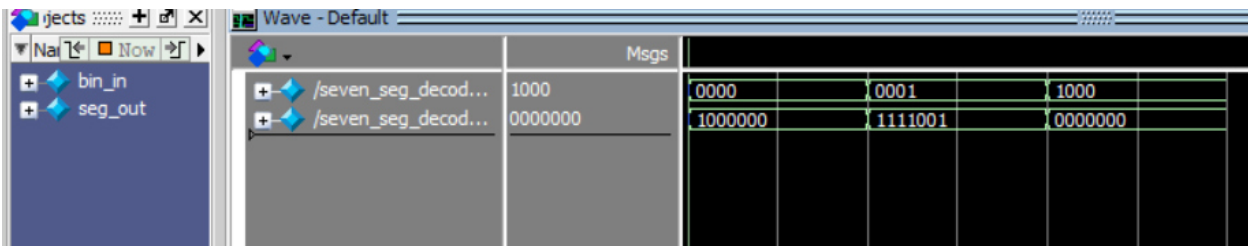
File seven_seg_decoder.sv

```
1 module seven_seg_decoder (
2     input logic [3:0] bin_in,
3     output logic [6:0] seg_out
4 );
5 // Active LOW
6 always_comb begin
7     case (bin_in)
8         4'h0: seg_out = 7'b1000000;
9         4'h1: seg_out = 7'b1111001;
10        4'h2: seg_out = 7'b0100100;
11        4'h3: seg_out = 7'b0110000;
12        4'h4: seg_out = 7'b0011001;
13        4'h5: seg_out = 7'b0010010;
14        4'h6: seg_out = 7'b0000010;
15        4'h7: seg_out = 7'b1111000;
16        4'h8: seg_out = 7'b0000000;
17        4'h9: seg_out = 7'b0010000;
18        default: seg_out = 7'b1111111;
19    endcase
20 end
21 endmodule
```

File seven_seg_decoder_tb.sv

```
1 module seven_seg_decoder_tb();
2     logic [3:0] bin_in;
3     logic [6:0] seg_out;
4
5     seven_seg_decoder dut (.*);
6
7     initial begin
8         $display("Testing 7-Seg Decoder...");
9         // 7'b1000000
10        bin_in = 4'd0; #10;
11        $display("In: %d -> Seg Out: %b", bin_in, seg_out);
12
13        // 7'b1111001
14        bin_in = 4'd1; #10;
15        $display("In: %d -> Seg Out: %b", bin_in, seg_out);
16
17        // 7'b0000000
18        bin_in = 4'd8; #10;
19        $display("In: %d -> Seg Out: %b", bin_in, seg_out);
20
21        $finish;
22    end
23 endmodule
```

Mô phỏng ModelSim:



Time	bin_in	seg_out
0000	0000	1000000
0001	0001	1111001
0002	1000	0000000

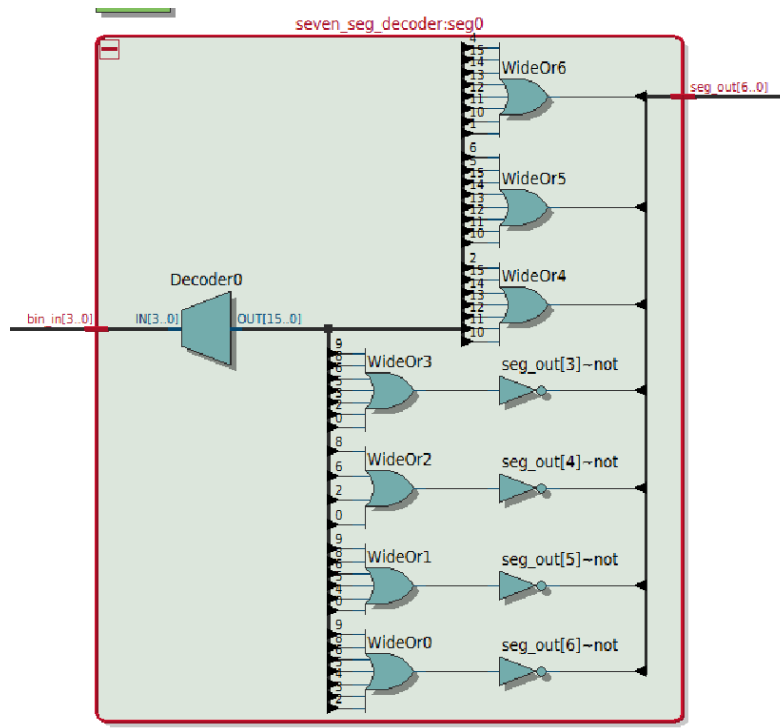
Testing 7-Seg Decoder...

In: 0 -> Seg Out: 1000000

In: 1 -> Seg Out: 1111001

In: 8 -> Seg Out: 0000000

Cấu trúc RTL Viewer thu được trên Quartus:



4.6. Module Khởi dữ liệu (VM_datapath)

- Đây là module phức tạp nhất, chứa đựng các thành phần lưu trữ thực tế.

+ Thanh ghi số dư (Balance Register): Lưu trữ giá trị tiền hiện có. Nó chỉ cập nhật khi tín hiệu `ld_balance` từ Controller bật cao.

+ Bộ định thời (Timer):

- Sử dụng một bộ đếm 32-bit: `timer_count`.
- Điều kiện reset: Khi có tiền mới nạp vào hoặc khi máy vừa nhả hàng thành công, Timer sẽ quay về 0 để khách hàng có đủ 15 giây mới.

+ Thuật toán thối tiền (Greedy Algorithm):

- Được triển khai bằng logic tổ hợp (`always_comb`).
- Sử dụng toán học để chia nhỏ số dư:

$$\text{ret_50} = \text{Balance} \pmod{50}$$

$$\text{rem_after_50} = \text{Balance} \pmod{50}$$

- Quá trình này lặp lại cho các mệnh giá 20, 10, 5 để đảm bảo số lượng tờ tiền trả lại là tối thiểu.

File VM_datapath.sv

```

1 module VM_datapath (
2     input logic clk, rst_n,
3
4     // Control Signals từ Controller
5     input logic ld_balance, clr_balance,
6     input logic ld_code, clr_code,
7     input logic alu_sub_mode,
8     input logic [1:0] src_sel,
9     input logic tmr_en, tmr_rst,
10    input logic calc_change,
11
12    // Data Inputs
13    input logic [7:0] coin_in,
14    input logic [4:0] key_num,
15
16    // Status Outputs
17    output logic bal_is_neg,
18    output logic bal_is_zero,
19    output logic code_is_zero,
20    output logic tmr_out,
21
22    // Data Outputs
23    output logic [7:0] current_balance,
24    output logic [7:0] price_value,
25    output logic [4:0] current_product_code,
26
27    // Outputs hiển thị số xu trả lại
28    output logic [3:0] ret_50, ret_20, ret_10, ret_5
29 );
30
31
32 // 1. Registers: Balance & Code
33 always_ff @(posedge clk or negedge rst_n) begin
34     if (!rst_n)
35         current_balance <= 8'd0;
36     else if (clr_balance)
37         current_balance <= 8'd0;
38     else if (ld_balance)
39         current_balance <= alu_out;
40     end
41
42 // Logic giữ mã hàng (Latch)
43 always_ff @(posedge clk or negedge rst_n) begin
44     if (!rst_n) begin
45         current_product_code <= 5'd0;
46     end else if (clr_code) begin
47         current_product_code <= 5'd0;
48     end else if (ld_code) begin
49         // Chỉ cập nhật nếu có phím nhấn (khác 0).
50         if (key_num != 5'd0) begin
51             current_product_code <= key_num;
52         end
53     end
54 end
55
56 // Tạo cờ báo code = 0 từ giá trị ĐÃ LƯU
57 assign code_is_zero = (current_product_code == 5'd0);
58
59 // 2. Price Lookup (Tra giá)
60 price_lookup price_mem (
61     .product_code(current_product_code),
62     .price(price_wire),
63     .is_invalid()
64 );
65 assign price_value = price_wire;
66

```

```

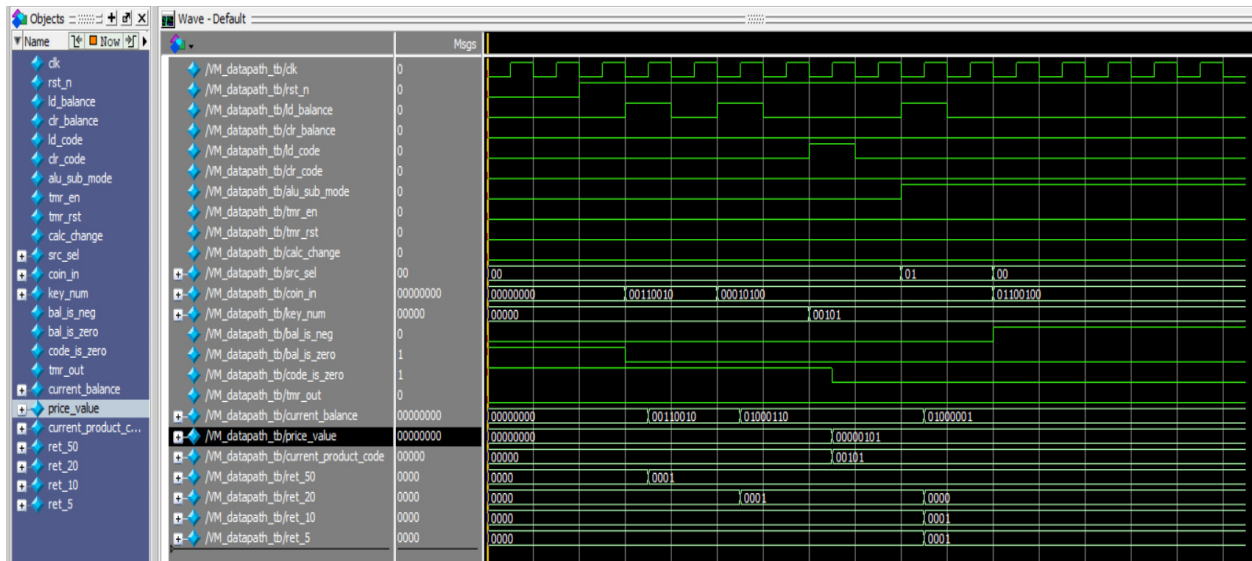
71 // 3. ALU Mux & Instantiation
72 always_comb begin
73     case (src_sel)
74         2'b00: alu_in_b = coin_in; // Nạp tiền
75         2'b01: alu_in_b = price_wire; // Trừ tiền mua
76         default: alu_in_b = 8'd0;
77     endcase
78 end
79
80 alu8 alu_inst (
81     .a(current_balance),
82     .b(alu_in_b),
83     .is_sub(alu_sub_mode),
84     .result(alu_out),
85     .is_neg(alu_neg_wire),
86     .is_zero(alu_zero_wire)
87 );
88
89 assign bal_is_neg = alu_neg_wire;
90 assign bal_is_zero = alu_zero_wire;
91
92 // 4. Timer Logic
93 localparam TIMEOUT_VAL = 15;
94 integer timer_count;
95
96 always_ff @(posedge clk or negedge rst_n) begin
97     if (!rst_n) begin
98         timer_count <= 0;
99         tmr_out <= 0;
100     end else begin
101         if (tmr_rst) begin
102             timer_count <= 0;
103             tmr_out <= 0;
104         end else if (tmr_en) begin
105             if (timer_count < TIMEOUT_VAL)
106                 timer_count <= timer_count + 1;
107             else
108                 tmr_out <= 1;
109         end else begin
110             timer_count <= 0;
111             tmr_out <= 0;
112         end
113     end
114 end
115
116 // 5. Greedy Algorithm for Change Return
117 logic [7:0] rem_after_50, rem_after_20, rem_after_10;
118
119 always_comb begin
120     ret_50 = 4'd0; ret_20 = 4'd0; ret_10 = 4'd0; ret_5 = 4'd0;
121     rem_after_50 = 8'd0; rem_after_20 = 8'd0; rem_after_10 = 8'd0;
122
123     // Luôn tính toán dựa trên current_balance (để sẵn sàng hiển thị khi cần)
124     ret_50 = current_balance / 50;
125     rem_after_50 = current_balance % 50;
126
127     ret_20 = rem_after_50 / 20;
128     rem_after_20 = rem_after_50 % 20;
129
130     ret_10 = rem_after_20 / 10;
131     rem_after_10 = rem_after_20 % 10;
132
133     ret_5 = rem_after_10 / 5;
134 end
135
136 endmodule

```

File VM_datapath_tb.sv

```
3 module VM_datapath_tb();
4     logic clk, rst_n;
5     logic ld_balance, clr_balance, ld_code, clr_code, alu_sub_mode, tmr_en, tmr_rst, calc_change;
6     logic [1:0] src_sel;
7     logic [7:0] coin_in;
8     logic [4:0] key_num;
9
10    // Outputs
11    logic bal_is_neg, bal_is_zero, code_is_zero, tmr_out;
12    logic [7:0] current_balance, price_value;
13    logic [4:0] current_product_code;
14    logic [3:0] ret_50, ret_20, ret_10, ret_5;
15
16    VM_datapath dut (.*);
17    always #5 clk = ~clk;
18
19    initial begin
20        // --- Khởi tạo ---
21        clk = 0; rst_n = 0;
22        ld_balance = 0; clr_balance = 0; ld_code = 0; clr_code = 0;
23        alu_sub_mode = 0; src_sel = 2'b00; tmr_en = 0; tmr_rst = 0; calc_change = 0;
24        coin_in = 0; key_num = 0;
25
26        #20 rst_n = 1;
27
28        // Case 1: Nạp tiền (Cộng dồn)
29        // Nạp 50k
30        #10 coin_in = 8'd50; ld_balance = 1; src_sel = 2'b00; alu_sub_mode = 0;
31        #10 ld_balance = 0;
32        // Nạp thêm 20k -> Tổng 70k
33        #10 coin_in = 8'd20; ld_balance = 1;
34        #10 ld_balance = 0;
35        $display("[Case 1] Balance after 50+20: %d", current_balance);
36
37        // Case 2: Tra giá và Trừ tiền (Mua hàng)
38        // Chọn sản phẩm số 5 (Giá sử giá 15k)
39        #10 key_num = 5'd5; ld_code = 1;
40        #10 ld_code = 0;
41        // Thực hiện phép trừ: 70 - 15 = 55
42        #10 src_sel = 2'b01; alu_sub_mode = 1; ld_balance = 1;
43        #10 ld_balance = 0;
44        $display("[Case 2] Price: %d, Balance after buy: %d", price_value, current_balance);
45
46        // Kịch bản 3: Thuật toán thối tiền (Greedy)
47        // Hiện tại balance = 55. Thối tiền: 50x1, 5x1
48        $display("[Case 3] Change: 50k:%d, 20k:%d, 10k:%d, 5k:%d", ret_50, ret_20, ret_10, ret_5);
49
50        // Kịch bản 4: Kiểm tra cò báo Âm (Thiếu tiền)
51        // Thù trừ tiếp 100k -> Balance 55 - 100 = âm
52        #10 coin_in = 8'd100; src_sel = 2'b00; alu_sub_mode = 1;
53        #5 if (bal_is_neg) $display("[Case 4] Success: bal_is_neg detected!");
54
55        #50 $finish;
56    end
57 endmodule
```

Mô phỏng ModelSim:



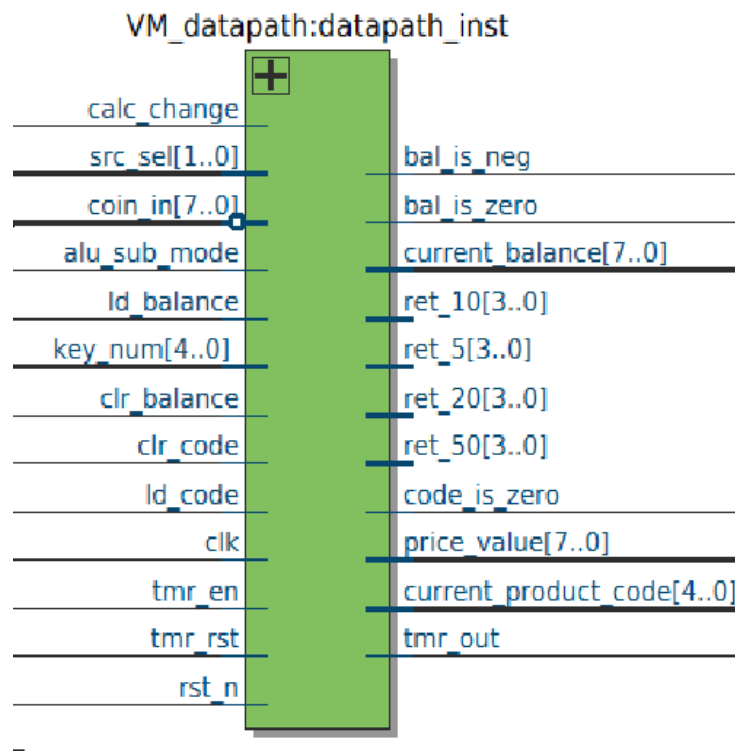
[Case 1] Balance after 50+20: 70

[Case 2] Price: 5, Balance after buy: 65

[Case 3] Change: 50k: 1, 20k: 0, 10k: 1, 5k: 1

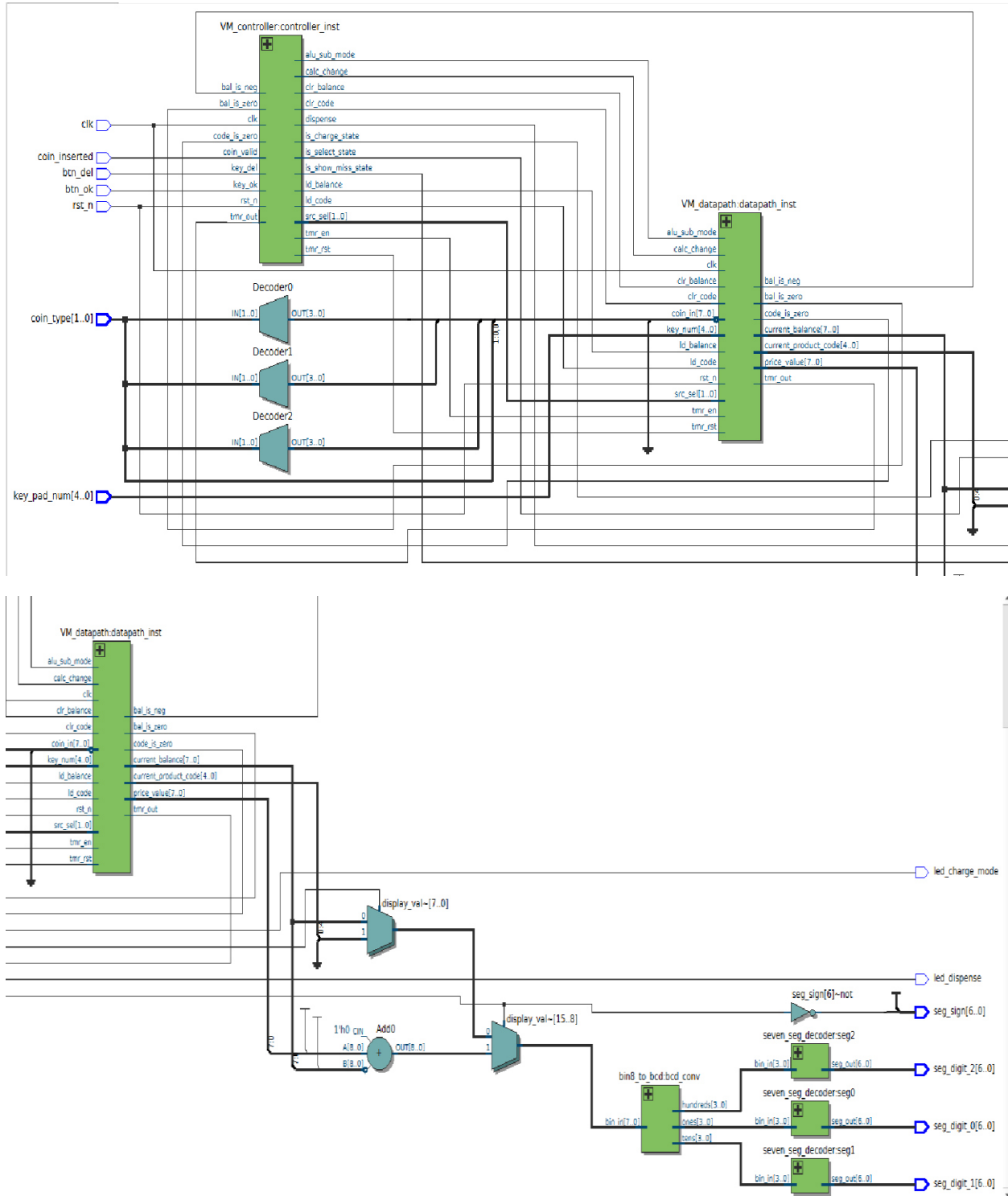
[Case 4] Success: `bal_is_neg` detected!

Cấu trúc RTL Viewer thu được trên Quartus:



IV) THIẾT KẾ KIẾN TRÚC TỔNG THỂ

4.1. Sơ đồ khối hệ thống của project



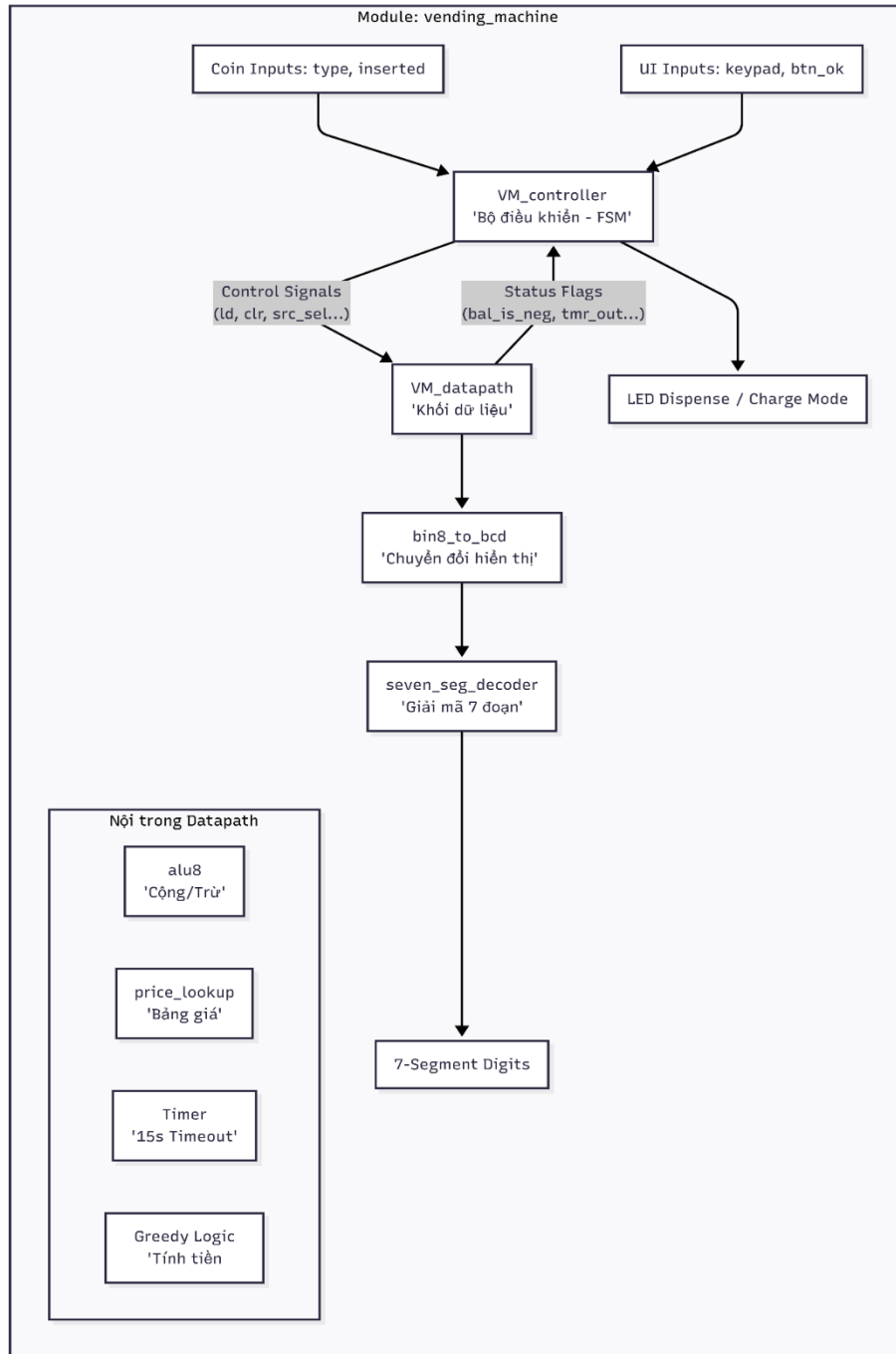
- Kiến trúc tổng thể của Project được xây dựng theo mô hình phân cấp (Hierarchy). Module mức đỉnh vending_machine đóng vai trò là "vỏ bọc" kết nối các thành phần chuyên biệt lại với nhau.

- Sơ đồ phân cấp module:

- **vending_machine (Top-level):** Quản lý kết nối IO và điều phối chung.
 - **VM_controller:** Khối logic điều khiển (FSM).
 - **VM_datapath:** Khối tính toán và lưu trữ dữ liệu.
 - alu8: Thực hiện cộng/trừ 8-bit.
 - price_lookup: Tra cứu giá từ bộ nhớ.
 - Timer: Đếm thời gian chờ.
 - **Hệ thống hiển thị:**
 - bin8_to_bcd: Chuyển đổi nhị phân sang số thập phân (BCD).
 - seven_seg_decoder: Giải mã 7 đoạn cho từng chữ số.

Sơ đồ luồng tín hiệu chính:

1. **Inputs (Coin/Keypad) → VM_datapath:** Dữ liệu thô đi vào khối tính toán.
2. **VM_datapath → VM_controller:** Các cờ trạng thái (hết tiền, thiếu tiền, hết thời gian) báo về bộ điều khiển.
3. **VM_controller → VM_datapath:** Các lệnh điều khiển (cho phép nạp tiền, trừ tiền, xóa dữ liệu) gửi ngược lại để thực thi.
4. **VM_datapath → BCD/Decoder → Segments:** Dữ liệu số dư/mã hàng được xuất ra màn hình người dùng.



4.2. Phân tích luồng thực thi lệnh

Luồng thực thi trong hệ thống không chạy theo dòng lệnh tuần tự như phần mềm (C/Python) mà chạy theo sự chuyển dịch trạng thái của FSM kết hợp với sự phản hồi của Datapath. Quy trình có thể chia thành 4 giai đoạn chính:

- Giai đoạn 1: Khởi tạo và Nạp tiền (IDLE & CHARGE)

- + Hệ thống bắt đầu tại S_IDLE. Khi người dùng nạp tiền, coin_type (2-bit) được giải mã thành giá trị 8-bit (5, 10, 20, 50).
- + VM_controller kích hoạt lệnh ld_balance. ALU thực hiện phép cộng và lưu giá trị mới vào thanh ghi current_balance.
- + Đồng thời, bộ đếm Timer bắt đầu hoạt động để quản lý thời gian chờ.
- Giai đoạn 2: Lựa chọn và Tra cứu (SELECT & CHECK)
 - + Khi người dùng nhập mã sản phẩm và nhấn btn_ok, mã này được lưu vào thanh ghi current_product_code.
 - + Khối price_lookup ngay lập tức trả về đơn giá (price_wire) tương ứng với mã đã chọn.
 - + Tại trạng thái S_CHECK, ALU thực hiện phép trừ "nháp": Balance - Price. Cờ bal_is_neg sẽ báo cho Controller biết tiền có đủ hay không.
- Giai đoạn 3: Xử lý kết quả (DISPENSE hoặc SHOW_MISS)
 - + Trường hợp đủ tiền: FSM chuyển sang S_DISPENSE. Tín hiệu led_dispense bật sáng (nhả hàng). Lệnh ld_balance được kích hoạt lần nữa để cập nhật số dư mới (đã trừ tiền hàng).
 - + Trường hợp thiếu tiền: FSM chuyển sang S_SHOW_MISS. Hệ thống sẽ tính toán số tiền thiếu bằng phép toán $\$Price - Balance\$$ và hiển thị lên LED 7 đoạn cùng dấu trừ (seg_sign).
- Giai đoạn 4: Trả tiền thừa và Kết thúc (RETURN)
 - + Nếu người dùng không thao tác sau 15 giây hoặc hoàn tất giao dịch và yêu cầu thoát, FSM chuyển sang S_RETURN.
 - + Lúc này, khối **Greedy Algorithm** trong Datapath sẽ hoạt động. Nó chia số dư còn lại cho các mệnh giá lớn nhất có thể ($50 \rightarrow 20 \rightarrow 10 \rightarrow 5$) để đưa ra số lượng tờ tiền cần trả lại qua các cổng ret_XX.
 - + Sau cùng, tín hiệu clr_balance và clr_code sẽ reset toàn bộ hệ thống về S_IDLE.

4.3 Top Module vending_machine

File vending_machine.sv

```
1 module vending_machine (
2     input logic clk,
3     input logic rst_n,
4
5     // UI Inputs
6     input logic [4:0] key_pad_num,
7     input logic btn_ok,
8     input logic btn_del,
9
10    // Money Inputs
11    input logic [1:0] coin_type,
12    input logic coin_inserted,
13
14    // Outputs
15    output logic [6:0] seg_digit_0,
16    output logic [6:0] seg_digit_1,
17    output logic [6:0] seg_digit_2,
18    output logic [6:0] seg_sign,
19    output logic led_dispense,
20    output logic led_charge_mode
21 );
22
23 // 1. Internal Signals
24 logic [7:0] coin_val_8bit;
25 logic coin_valid_trigger;
26
27 // Tín hiệu Status
28 logic dp_bal_neg, dp_bal_zero, dp_code_is_zero, dp_tmr_out;
29
30 // Tín hiệu Control
31 logic ctrl_ld_bal, ctrl_clr_bal;
32 logic ctrl_ld_code, ctrl_clr_code;
33 logic ctrl_alu_sub;
34 logic [1:0] ctrl_src_sel;
35 logic ctrl_tmr_en, ctrl_tmr_rst, ctrl_calc_change;
36
37 // Status Flags cho hiển thị
38 logic ctrl_show_miss;
39 logic ctrl_select_state;
40 logic ctrl_charge_state;
41 logic ctrl_return_state;
42
43 // Data wires
44 logic [7:0] dp_balance;
45 logic [7:0] dp_price;
46 logic [4:0] dp_code;
47
48 logic [3:0] ret_50, ret_20, ret_10, ret_5;
49
50 // 2. Xử lý đầu vào Coin
51 always_comb begin
52     case (coin_type)
53         2'b00: coin_val_8bit = 8'd5;
54         2'b01: coin_val_8bit = 8'd10;
55         2'b10: coin_val_8bit = 8'd20;
56         2'b11: coin_val_8bit = 8'd50;
57         default: coin_val_8bit = 8'd0;
58     endcase
59 end
60 assign coin_valid_trigger = coin_inserted;
61
62 // 3. Instantiate Controller
63 VM_controller controller_inst (
64     .clk(clk),
65     .rst_n(rst_n),
66     .coin_valid(coin_valid_trigger),
67     .key_ok(btn_ok),
68     .key_del(btn_del),
69
70     .bal_is_neg(dp_bal_neg),
71     .bal_is_zero(dp_bal_zero),
72     .code_is_zero(dp_code_is_zero),
73     .tmr_out(dp_tmr_out),
74
```

```

75         .ld_balance(ctrl_ld_bal),
76         .clr_balance(ctrl_clr_bal),
77         .ld_code(ctrl_ld_code),
78         .clr_code(ctrl_clr_code),
79         .alu_sub_mode(ctrl_alu_sub),
80         .src_sel(ctrl_src_sel),
81
82         .tmr_en(ctrl_tmr_en),
83         .tmr_rst(ctrl_tmr_rst),
84         .calc_change(ctrl_calc_change),
85
86         .dispense(led_dispense),
87         .is_show_miss_state(ctrl_show_miss),
88         .is_select_state(ctrl_select_state),
89         .is_charge_state(ctrl_charge_state),
90         .is_return_state(ctrl_return_state)
91     );
92
93     assign led_charge_mode = ctrl_charge_state;
94
95     // 4. Instantiate Datapath
96     VM_datapath datapath_inst (
97         .clk(clk),
98         .rst_n(rst_n),
99         .ld_balance(ctrl_ld_bal),
100        .clr_balance(ctrl_clr_bal),
101        .ld_code(ctrl_ld_code),
102        .clr_code(ctrl_clr_code),
103        .alu_sub_mode(ctrl_alu_sub),
104        .src_sel(ctrl_src_sel),
105        .tmr_en(ctrl_tmr_en),
106        .tmr_rst(ctrl_tmr_rst),
107        .calc_change(ctrl_calc_change),
108        .coin_in(coin_val_8bit),
109        .key_num(key_pad_num),
110        .bal_is_neg(dp_bal_neg),
111        .bal_is_zero(dp_bal_zero),
112        .code_is_zero(dp_code_is_zero),
113        .tmr_out(dp_tmr_out),
114        .current_balance(dp_balance),
115        .price_value(dp_price),
116        .current_product_code(dp_code),
117        .ret_50(ret_50), .ret_20(ret_20),
118        .ret_10(ret_10), .ret_5(ret_5)
119    );
120
121     // 5. Logic Hiển thị
122     logic [7:0] display_val;
123     logic [3:0] bcd_h, bcd_t, bcd_o;
124
125     always_comb begin
126         // Mặc định
127         seg_sign = 7'b1111111;
128
129         if (ctrl_show_miss) begin
130             // SHOW_MISS: Price - Balance (Số tiền thiếu)
131             display_val = dp_price - dp_balance;
132             seg_sign = 7'b0111111; // Dấu trừ (-)
133         end
134         else if (ctrl_select_state) begin
135             // SELECT: Mã hàng (Hiển thị giá trị ĐÃ LƯU dp_code)
136             display_val = {3'b000, dp_code};
137         end
138         else if (ctrl_return_state) begin
139             // RETURN: Số tiền trả lại
140             display_val = dp_balance;
141         end
142         else begin
143             // CHARGE, IDLE, DISPENSE: Hiển thị số dư hiện tại
144             display_val = dp_balance;
145         end
146     end
147
148     // Module đổi Bin -> BCD và Decoder (Giữ nguyên)
149     bin8_to_bcd bcd_conv (
150         .bin_in(display_val),
151         .hundreds(bcd_h), .tens(bcd_t), .ones(bcd_o)
152     );
153     seven_seg_decoder seg0 (.bin_in(bcd_o), .seg_out(seg_digit_0));
154     seven_seg_decoder seg1 (.bin_in(bcd_t), .seg_out(seg_digit_1));
155     seven_seg_decoder seg2 (.bin_in(bcd_h), .seg_out(seg_digit_2));
156
157 endmodule

```

File vending_machine_tb.sv

```

1  `timescale 1ns/1ps
2
3  module vending_machine_tb;
4      logic clk;
5      logic rst_n;
6
7      // Inputs
8      logic [4:0] key_pad_num;
9      logic btn_ok;
10     logic btn_del;
11     logic [1:0] coin_type;    // 00:5, 01:10, 10:20, 11:50
12     logic coin_inserted;
13
14     // Outputs
15     logic [6:0] seg_digit_0, seg_digit_1, seg_digit_2, seg_sign;
16     logic led_dispense;
17     logic led_charge_mode;
18
19     vending_machine uut (
20         .clk(clk),
21         .rst_n(rst_n),
22         .key_pad_num(key_pad_num),
23         .btn_ok(btn_ok),
24         .btn_del(btn_del),
25         .coin_type(coin_type),
26         .coin_inserted(coin_inserted),
27         .seg_digit_0(seg_digit_0),
28         .seg_digit_1(seg_digit_1),
29         .seg_digit_2(seg_digit_2),
30         .seg_sign(seg_sign),
31         .led_dispense(led_dispense),
32         .led_charge_mode(led_charge_mode)
33     );
34
35     always #5 clk = ~clk;
36
37     // Biền cờ để bắt xung dispense
38     logic dispense_occurred;
39
40     always @(posedge clk or negedge rst_n) begin
41         if (!rst_n) dispense_occurred <= 0;
42         else if (led_dispense) dispense_occurred <= 1;
43     end
44
45     always @(uut.controller_inst.current_state) begin
46         $display("Time %0t: [FSM] State changed to: %s", $time, uut.controller_inst.current_state.name());
47     end
48
49     //Helper Tasks
50     task insert_coin(input [1:0] coin_val_in);
51     begin
52         @(posedge clk);
53         coin_type = coin_val_in;
54         coin_inserted = 1;
55         @(posedge clk);
56         coin_inserted = 0;
57         repeat(2) @(posedge clk);
58     end
59     endtask
60
61     task press_num(input [4:0] num);
62     begin
63         @(posedge clk);
64         key_pad_num = num;
65         repeat(2) @(posedge clk);
66         key_pad_num = 0;
67         repeat(2) @(posedge clk);
68     end
69     endtask
70
71     task press_ok_btn();
72     begin
73         @(posedge clk);
74         btn_ok = 1;
75         repeat(2) @(posedge clk);
76         btn_ok = 0;
77         repeat(2) @(posedge clk);
78     end
79     endtask
80

```

```

81 task press_del_btn();
82 begin
83     @(posedge clk);
84     btn_del = 1;
85     repeat(2) @(posedge clk);
86     btn_del = 0;
87     repeat(2) @(posedge clk);
88 end
89 endtask
90
91 // Main Test Sequence
92 initial begin
93     // Initialize
94     clk = 0; rst_n = 0;
95     key_pad_num = 0; btn_ok = 0; btn_del = 0;
96     coin_type = 0; coin_inserted = 0;
97
98     // Reset system
99     #20 rst_n = 1;
100    $display("-----");
101    $display("START SIMULATION");
102    $display("-----");
103
104    // CASE 1: Mua hàng đủ tiền
105    $display("\n[CASE 1] Normal Buy: Insert 5, Buy Item 1 (Price 5)");
106    dispense_occurred = 0;
107
108    insert_coin(2'b00);
109    press_ok_btn();
110    press_num(5'd1);
111    press_ok_btn();
112
113    repeat(5) @(posedge clk);
114
115    if(dispense_occurred) $display(">>> PASS: Item Dispensed Successfully!");
116    else $display(">>> FAIL: Item NOT Dispensed");
117
118    repeat(5) @(posedge clk); // Về IDLE
119
120    // CASE 2: Thiếu tiền
121    $display("\n[CASE 2] Not Enough Money: Insert 5, Buy Item 6 (Price 10)");
122
123    insert_coin(2'b00); // Nạp thêm 5
124    press_ok_btn(); // vào SELECT
125    press_num(5'd6); // Chọn mã 6 (Giá 10)
126    press_ok_btn(); // Mua -> Thiếu tiền -> Show Miss
127
128    repeat(3) @(posedge clk);
129    if (seg_sign == 7'b0111111) $display(">>> PASS: SHOW_MISS detected (Negative Sign ON)");
130    else $display(">>> FAIL: Not in SHOW_MISS state");
131
132    // Quay lại nạp tiền
133    press_ok_btn();
134    $display("Pressed OK -> Back to CHARGE");
135
136    dispense_occurred = 0;
137    insert_coin(2'b00); // Nạp thêm 5 (Tổng 10)
138    press_ok_btn(); // Vào lại SELECT
139    press_num(5'd6); // Chọn lại mã 6
140    press_ok_btn(); // Mua
141
142    repeat(5) @(posedge clk);
143    if(dispense_occurred) $display(">>> PASS: Added money & Item Dispensed!");
144    else $display(">>> FAIL: Item NOT Dispensed after adding money");
145
146    repeat(5) @(posedge clk);
147
148    // CASE 3: Mua nhiều món
149    $display("\n[CASE 3] Multi-Buy: Insert 20, Buy Item 1 (Price 5)");
150    dispense_occurred = 0;
151
152    insert_coin(2'b10); // Nạp 20
153    press_ok_btn();
154    press_num(5'd1); // Mua món 5
155    press_ok_btn();
156
157    repeat(5) @(posedge clk);
158    if(dispense_occurred) $display(">>> PASS: First item dispensed");
159
160    $display("Checking if system stays in SELECT (Balance > 0)...");
161    dispense_occurred = 0;
162    press_num(5'd2); // Chọn mã 2 (Giá 5)
163    press_ok_btn();

```



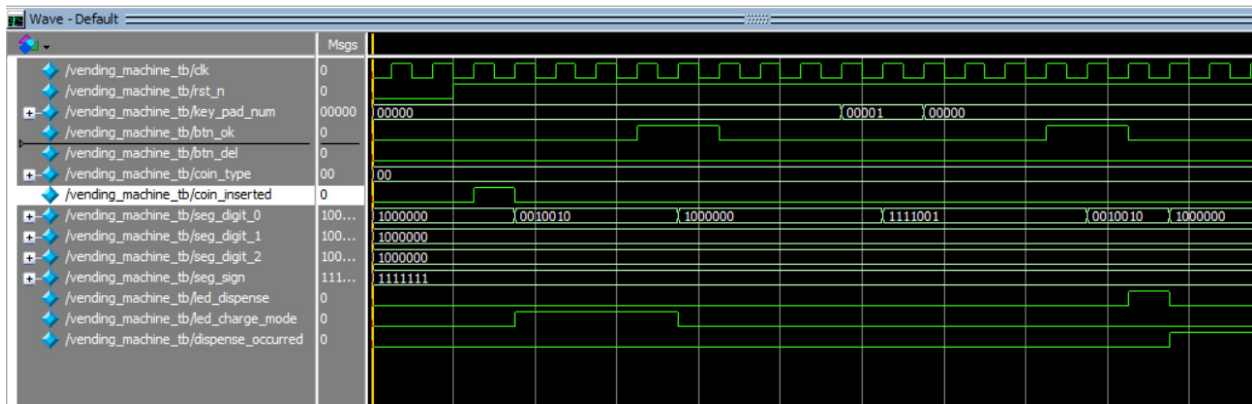
```

165         repeat(5) @(posedge clk);
166         if(dispense_occurred) $display(">>> PASS: Second item dispensed with remaining balance!");
167
168         $display("Refunding remaining 10...");
169         press_ok_btn();
170         repeat(5) @(posedge clk);
171
172         // CASE 4: TIMEOUT
173         $display("\n[CASE 4] Timeout Test");
174
175         insert_coin(2'b01); // Nap 10
176         $display("Inserted 10. Waiting for timeout...");
177
178         repeat(20) @(posedge clk);
179
180         if (led_charge_mode == 0) $display(">>> PASS: Returned to IDLE automatically");
181         else $display(">>> FAIL: Still in Charge Mode");
182
183         // CASE 5: DELETE BUTTON (Back to Charge)
184         $display("\n[CASE 5] Test DELETE Button (Back to Charge)");
185
186         insert_coin(2'b00); // Nap 5
187         press_ok_btn(); // Vào SELECT
188
189         $display("Currently in SELECT. Pressing DEL...");
190         press_del_btn();
191
192         repeat(2) @(posedge clk);
193         if (led_charge_mode == 1) $display(">>> PASS: Returned to CHARGE mode");
194         else $display(">>> FAIL: Did not return to CHARGE");
195
196         $display("\n-----");
197         $display("ALL TESTS COMPLETED");
198         $display("-----");
199         $stop;
200     end
201 endmodule

```

Mô Phỏng ModelSim:

- Testcase 1: Mua hàng đủ tiền



```

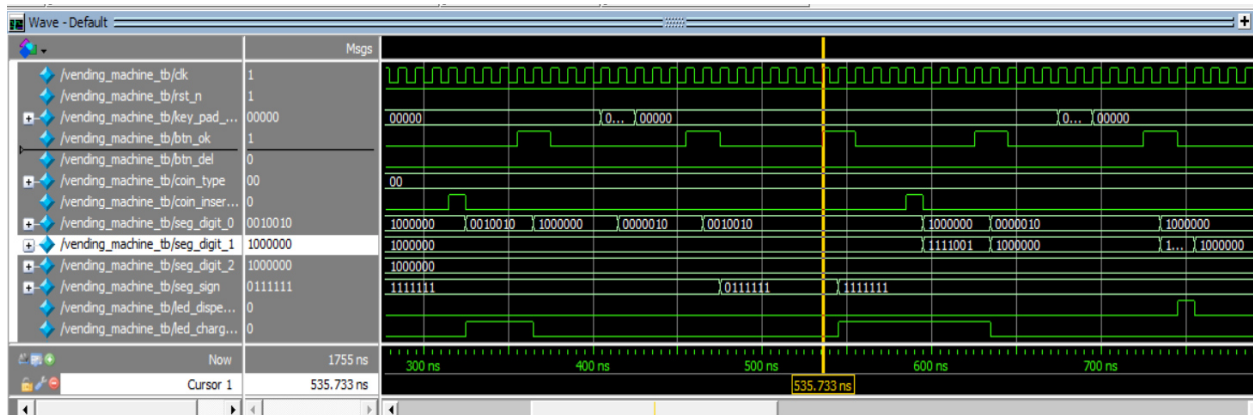
# [CASE 1] Normal Buy: Insert 5, Buy Item 1 (Price 5)
# Time 35000: [FSM] State changed to: S_CHARGE
# Time 75000: [FSM] State changed to: S_SELECT
# Time 175000: [FSM] State changed to: S_CHECK
# Time 185000: [FSM] State changed to: S_DISPENSE
# Time 195000: [FSM] State changed to: S_RETURN
# Time 205000: [FSM] State changed to: S_IDLE
# >>> PASS: Item Dispensed Successfully!

```

+ Giải thích hoạt động:

- IDLE: Cho tiền vào (5k), coin_inserted sẽ từ mức 0 → mức 1. Trên leg 7 đoạn seg_digit_0 sẽ hiển thị giá trị thập phân 5. Trạng thái lúc này chuyển từ IDLE → CHARGE (leg_charge_mode từ mức 0 → mức 1)
- CHARGE: Ở trạng thái này, người dùng sẽ có thể tiếp tục cho tiền của mình vào hoặc nhấn OK để chuyển sang trạng thái SELECT
- SELECT: Chọn sản phẩm số 1, key_pad_num sẽ nhận tín hiệu nhị phân (00001) đồng thời leg_digit_0 cũng hiển thị giá trị thập phân của sản phẩm đó (1). Nhấn OK để chuyển sang trạng thái CHECK
- CHECK: Tổng giá trị hàng hoá được hiển thị trên các leg 7 đoạn (leg_digit_0 = 0010010 tức tổng giá trị hàng hoá là 5k). Khi đã kiểm tra và nhận thấy đủ tiền (Số thập phân hiển thị trên leg 7 đoạn bằng 0), máy sẽ chuyển sang trạng thái DISPENSE
- DISPENSE: Thanh leg_dispense sẽ từ mức 0 → 1. Máy sẽ thực hiện nhả hàng đồng thời chuyển sang trạng thái RETURN
- RETURN: vì tiền thừa bằng 0 nên sẽ lập tức trở về trạng thái IDLE ban đầu

- Testcase 2: Thiếu tiền

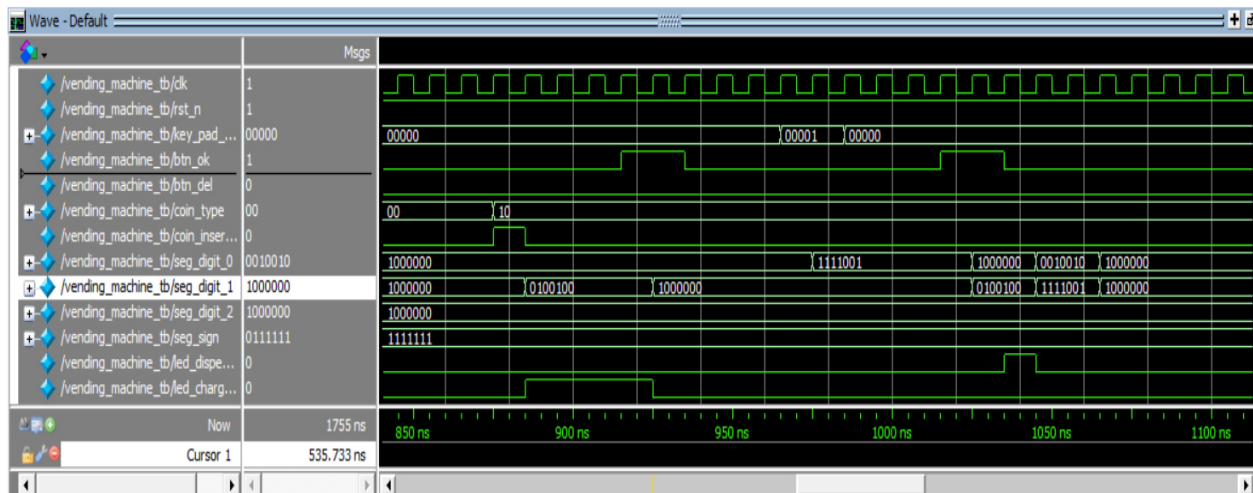


```
#
# [CASE 2] Not Enough Money: Insert 5, Buy Item 6 (Price 10)
# Time 325000: [FSM] State changed to: S_CHARGE
# Time 365000: [FSM] State changed to: S_SELECT
# Time 465000: [FSM] State changed to: S_CHECK
# Time 475000: [FSM] State changed to: S_SHOW_MISS
# >>> PASS: SHOW_MISS detected (Negative Sign ON)
# Time 545000: [FSM] State changed to: S_CHARGE
# Pressed OK -> Back to CHARGE
# Time 635000: [FSM] State changed to: S_SELECT
# Time 735000: [FSM] State changed to: S_CHECK
# Time 745000: [FSM] State changed to: S_DISPENSE
# Time 755000: [FSM] State changed to: S_RETURN
# Time 765000: [FSM] State changed to: S_IDLE
# >>> PASS: Added money & Item Dispensed!
```

+ Giải thích hoạt động:

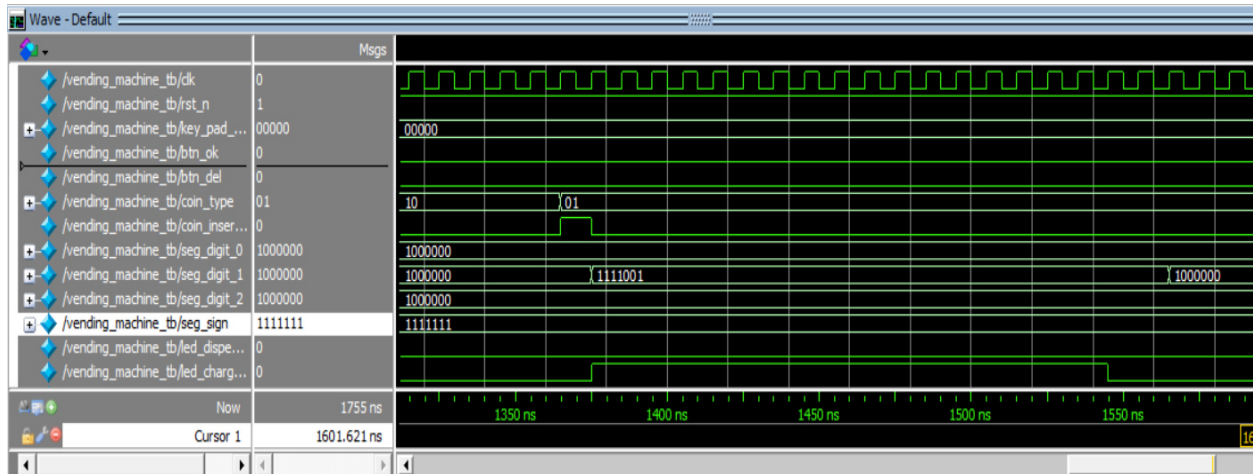
- IDLE: Cho tiền vào (5k). Trên leg 7 đoạn seg_digit_0 sẽ hiển thị giá trị thập phân 5. Trạng thái lúc này chuyển từ IDLE → CHARGE → SELECT
- SELECT: Chọn sản phẩm số 6, key_pad_num sẽ nhận tín hiệu nhị phân đồng thời leg_digit_0 cũng hiển thị giá trị thập phân của sản phẩm đó (6). Nhấn OK để chuyển sang trạng thái CHECK
- CHECK: Tổng tiền bỏ vào - Tổng giá trị hàng hoá < 0. Trạng thái lúc này chuyển sang SHOW_MISS
- SHOW_MISS: Kết quả được hiển thị trên leg là -5k (leg_digit_0 là 0010010 và seg_sign là tính hiệu biểu diễn giá trị âm đã được bật). Khi 5k được bỏ vào lần 2, trạng thái chuyển thành CHARGE → SELECT.
- SELECT: Chọn lại sản phẩm 6. Trạng thái được chuyển sang CHECK.
- CHECK: Đã đủ tiền để mua sản phẩm, kích hoạt DISPENSE và RETURN sau đó trở về trạng thái IDLE

- Testcase 3: Mua nhiều món



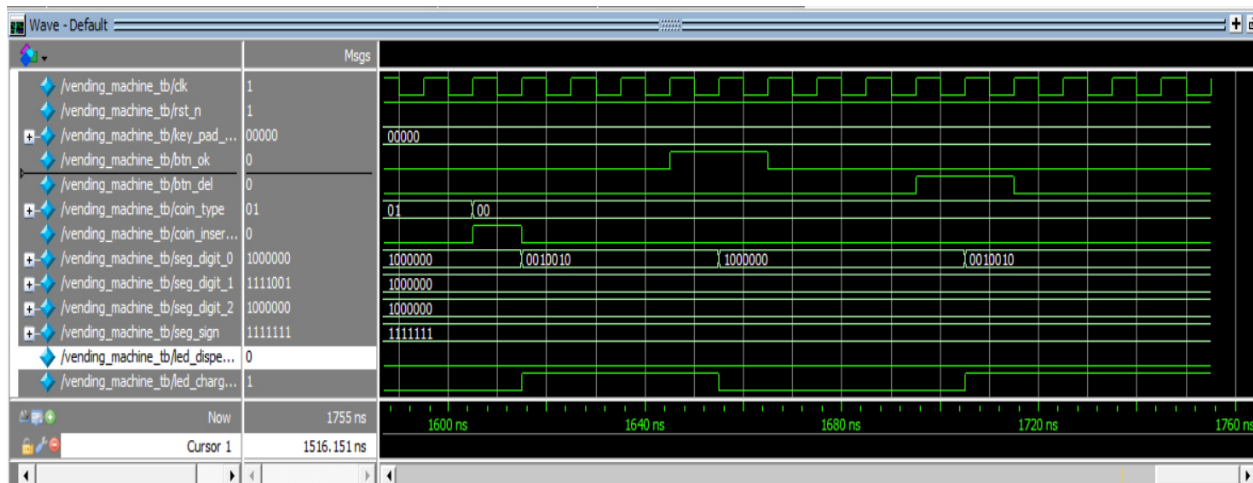
```
# [CASE 3] Multi-Buy: Insert 20, Buy Item 1 (Price 5)
# Time 885000: [FSM] State changed to: S_CHARGE
# Time 925000: [FSM] State changed to: S_SELECT
# Time 1025000: [FSM] State changed to: S_CHECK
# Time 1035000: [FSM] State changed to: S_DISPENSE
# Time 1045000: [FSM] State changed to: S_RETURN
# Time 1055000: [FSM] State changed to: S_IDLE
# >>> PASS: First item dispensed
# Checking if system stays in SELECT (Balance > 0)...
# Refunding remaining 10...
```


- Testcase 4: Nạp tiền vào nhưng không mua



```
#
# [CASE 4] Timeout Test
# Time 1375000: [FSM] State changed to: S_CHARGE
# Inserted 10. Waiting for timeout...
# Time 1545000: [FSM] State changed to: S_RETURN
# Time 1555000: [FSM] State changed to: S_IDLE
# >>> PASS: Returned to IDLE automatically
#
```

- Testcase 5: Hủy chọn hàng hoặc sửa đổi giao dịch bằng nút bấm DELETE (btn_del)



```
# [CASE 5] Test DELETE Button (Back to Charge)
# Time 1615000: [FSM] State changed to: S_CHARGE
# Time 1655000: [FSM] State changed to: S_SELECT
# Currently in SELECT. Pressing DEL...
# Time 1705000: [FSM] State changed to: S_CHARGE
# >>> PASS: Returned to CHARGE mode
#
```